Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

École Internationale Supérieure Privée Polytechnique de Sousse

RAPPORT DE

PROJET D'INTÉGRATION

Détection automatique des discours haineux et abusifs en dialecte tunisien par apprentissage profond

---

Réalisé par : Salma Mlika

Encadré par : M. Mehdi Selem

Année Universitaire 2025 – 2026

Résumé

Ce projet vise à développer un système de classification automatique de textes en dialecte tunisien utilisant l'apprentissage profond. Face à la prolifération du contenu en ligne et la nécessité de modérer les discours haineux et abusifs, nous avons fine-tuné le modèle BERT pré-entraîné AraBERT sur le dataset T-HSAB. Notre système classe les textes en trois catégories : normal, abusif et haineux. Pour gérer le déséquilibre des classes, nous avons appliqué une technique de downsampling de la classe majoritaire. Les résultats montrent une précision satisfaisante avec un F1-score pondéré démontrant l'efficacité de l'approche par transfert d'apprentissage pour les langues peu dotées comme le dialecte tunisien.

Mots-clés : Traitement automatique du langage naturel, BERT, AraBERT, dialecte tunisien, classification de textes, discours haineux, apprentissage profond, downsampling

Table des matières

Résumé **i**

Introduction Générale **1**

1 Contexte et Environnement du Projet **3**

1.1 Contexte général 3

1.2 Technologies et outils utilisés 3

1.2.1 Environnement de développement 3

1.2.2 Bibliothèques et frameworks 3

1.2.3 Modèle pré-entraîné 3

1.3 Dataset T-HSAB 4

1.3.1 Présentation du dataset 4

1.3.2 Caractéristiques du dialecte tunisien 4

1.3.3 Distribution initiale des données 4

1.3.4 Distribution après downsampling 5

2 Étude de Marché et État de l'Art **6**

2.1 Analyse du marché 6

2.1.1 Contexte du marché 6

2.1.2 Opportunités et défis 6

2.2 État de l'art 6

2.2.1 Traitement automatique du langage naturel 6

2.2.2 BERT et ses variantes 7

2.2.3 Détection de discours haineux 7

2.2.4 Gestion du déséquilibre des classes 7

2.2.5 Travaux sur les dialectes arabes 7

2.3 Positionnement du projet 7

3 Cadre Théorique et Analytique 9

3.1 Concepts fondamentaux 9

3.1.1 Apprentissage automatique supervisé 9

3.1.2 Architecture Transformer 9

3.1.3 BERT en détail 9

3.2 Gestion du déséquilibre des classes 10

3.2.1 Problématique du déséquilibre 10

3.2.2 Technique de downsampling 10

3.3 Méthodologie 10

3.3.1 Pipeline de traitement 10

3.3.2 Prétraitement des données 11

3.3.3 Tokenisation 11

3.3.4 Fine-tuning 11

3.4 Métriques d'évaluation 11

3.4.1 Accuracy 11

3.4.2 Précision et Recall 11

3.4.3 F1-Score 12

4 Implémentation et Mise en Pratique 13

4.1 Exploration des données 13

4.1.1 Chargement du dataset 13

4.1.2 Analyse exploratoire 13

4.2 Gestion du déséquilibre des classes 14

4.2.1 Analyse du déséquilibre initial 14

4.2.2 Application du downsampling 14

4.2.3 Impact attendu du downsampling 15

4.3 Prétraitement et préparation des données 15

4.3.1 Nettoyage 15

4.3.2 Division des données 15

4.4 Configuration du modèle 16

4.4.1 Tokenisation 16

4.4.2 Préparation des DataLoaders 17
4.4.3 Initialisation du modèle 17
4.5 Entraînement du modèle 18
4.5.1 Boucle d'entraînement 18
4.5.2 Évolution de la loss 19
4.6 Évaluation et résultats 19
4.6.1 Protocole d'évaluation 19
4.6.2 Métriques de performance 20
4.6.3 Tests qualitatifs 20
4.7 Analyse comparative et impact du downsampling 20
4.7.1 Comparaison avant/après downsampling 20
4.7.2 Amélioration significative de la détection de la classe "abusive" 21
4.7.3 Performance sur la classe "hate" 21
4.7.4 Trade-off avec la classe "normal" 22
4.7.5 Analyse de l'accuracy globale 22
4.7.6 Validation qualitative 22
4.7.7 Recommandations pratiques 23
4.7.8 Limites de l'évaluation actuelle 23
4.8 Analyse critique 23
4.8.1 Points forts 23
4.8.2 Limitations 24
4.8.3 Perspectives d'amélioration 24

Conclusion Générale **25**

Bibliographie **27**

A Code source complet **29**
A.1 Imports et configuration 29
A.2 Fonction de nettoyage 29
A.3 Pipeline complet 30

Liste des tableaux

1.1 Principales bibliothèques utilisées 4
1.2 Distribution initiale des classes dans le dataset 5
1.3 Distribution des classes après downsampling 5

4.1 Métriques de performance du modèle avec downsampling 20
4.2 Exemples de tests et prédictions du modèle après downsampling 20
4.3 Comparaison des performances avant et après downsampling 21

Introduction Générale

Le traitement automatique du langage naturel (TALN) connaît une révolution grâce aux modèles de transformateurs tels que BERT. Cependant, la majorité des recherches se concentrent sur les langues largement dotées comme l'anglais ou le français standard. Les dialectes arabes, et particulièrement le dialecte tunisien, demeurent sous-représentés malgré leur utilisation massive sur les réseaux sociaux et plateformes en ligne.

La modération du contenu en ligne est devenue un enjeu majeur avec l'augmentation des discours haineux et abusifs. Les entreprises et institutions ont besoin d'outils automatiques capables de détecter et classifier ces contenus pour protéger les utilisateurs et maintenir un environnement sain en ligne.

Problématique

Comment développer un système de classification automatique efficace pour identifier les discours haineux et abusifs dans les textes en dialecte tunisien, en exploitant les techniques d'apprentissage profond par transfert et en gérant le déséquilibre des classes ?

Objectifs

Les objectifs principaux de ce projet sont les suivants. Premièrement, étudier les architectures de transformateurs et leur application au dialecte tunisien. Deuxièmement, gérer le déséquilibre des classes dans le dataset T-HSAB par des techniques de rééchantillonnage. Troisièmement, fine-tuner le modèle AraBERT sur un dataset équilibré en dialecte tunisien. Quatrièmement, développer un système de classification en trois catégories : normal, abusif et haineux. Enfin, évaluer les performances du modèle et analyser l'impact du downsampling sur les résultats obtenus.

Plan du rapport

Ce rapport s'articule autour de quatre chapitres. Le premier présente le contexte du projet et les technologies utilisées. Le deuxième expose l'état de l'art en matière de TALN et de détection de discours haineux. Le troisième détaille le cadre théorique et méthodologique incluant la gestion du déséquilibre des classes. Enfin, le quatrième chapitre présente l'implémentation, les résultats et leur analyse comparative.

Chapitre 1

Contexte et Environnement du Projet

1.1 Contexte général

Ce projet s'inscrit dans le cadre du projet d'intégration à l'Ecole Internationale Supérieure Privée Polytechnique de Sousse, sous la supervision de Monsieur Mehdi Selem. Il vise à appliquer les connaissances acquises en intelligence artificielle et apprentissage automatique à un problème réel de traitement du langage naturel.

1.2 Technologies et outils utilisés

1.2.1 Environnement de développement

Le développement du projet a été réalisé dans l'environnement Google Colab, offrant l'accès à des GPU pour accélérer l'entraînement des modèles de deep learning. Python 3.x a été utilisé comme langage de programmation principal.

1.2.2 Bibliothèques et frameworks

Les principales bibliothèques utilisées sont présentées dans le tableau 1.1.

1.2.3 Modèle pré-entraîné

Nous avons utilisé AraBERT (aubmindlab/bert-base-arabert), un modèle BERT pré-entraîné spécifiquement sur des textes arabes par l'équipe AUB MIND Lab. Ce modèle comprend mieux les spécificités linguistiques de l'arabe et ses dialectes.

Tab. 1.1 : Principales bibliothèques utilisées

Bibliothèque	**Description**
PyTorch	Framework de deep learning pour la construction et l'entraînement des modèles
Transformers	Bibliothèque Hugging Face fournissant des modèles pré-entraînés comme BERT
Scikit-learn	Outils pour l'évaluation des performances, la division des données et le rééchantillonnage
Pandas	Manipulation et analyse des données
Matplotlib	Visualisation des données

1.3 Dataset T-HSAB

1.3.1 Présentation du dataset

Le dataset utilisé est T-HSAB (Tunisian Hate Speech and Abusive Language), le premier corpus annoté pour la détection de discours haineux et abusifs en dialecte tunisien. Ce dataset a été développé par Hatem Haddad,Hala Mulki et Asma Oueslati1 de l'Université de Manouba en Tunisie et publié suite à la révolution de jasmin de 2011, période qui a marqué une nouvelle ère de liberté d'expression en Tunisie.

Le dataset combine 6,039 commentaires collectés sur les réseaux sociaux tunisiens et annotés selon trois catégories : Normal, Abusive et Hate. Les annotations ont été réalisées suivant des guidelines stricts avec validation par calcul du Cohen's Kappa et Krippendorff's alpha pour assurer la cohérence.

1.3.2 Caractéristiques du dialecte tunisien

Le dialecte tunisien, aussi connu sous le nom de "Tounsi" ou "Derja", présente des particularités importantes. Il diffère significativement de l'arabe standard moderne. Il intègre du vocabulaire arabe mélangé avec des mots et phrases de l'amazigh, du français, du turc, de l'italien et d'autres langues. Cette richesse linguistique rend la tâche de classification particulièrement complexe et intéressante.

1.3.3 Distribution initiale des données

Notre échantillon de travail extrait du dataset T-HSAB contient 997 exemples après suppression des doublons. La distribution initiale des classes montre un déséquilibre important présenté

dans le tableau 1.2.

Tab. 1.2 : Distribution initiale des classes dans le dataset

Classe	**Nombre d'exemples**	**Pourcentage**
Normal	616	61.8%
Abusive	212	21.3%
Hate	172	17.2%
Total	**997**	**100%**

Cette distribution révèle un déséquilibre significatif avec la classe "Normal" représentant 61.8% du dataset, soit près de trois fois plus que la classe "Hate" (17.2%).

1.3.4 Distribution après downsampling

Après application du downsampling sur la classe majoritaire "Normal", la distribution devient équilibrée comme montré dans le tableau 1.3.

Tab. 1.3 : Distribution des classes après downsampling

Classe	**Nombre d'exemples**	**Pourcentage**
Normal	172	30.9%
Abusive	212	38.1%
Hate	172	30.9%
Total	**556**	**100%**

Le downsampling a ramené la classe "Normal" au même niveau que la classe "Hate" (172 exemples chacune), créant ainsi un dataset beaucoup plus équilibré. La taille totale du dataset est passée de 997 à 556 exemples, soit une réduction de 44.2%.

Chapitre 2

Étude de Marché et État de l'Art

2.1 Analyse du marché

2.1.1 Contexte du marché

La modération de contenu en ligne représente un marché en forte croissance, estimé à plusieurs milliards de dollars. Les plateformes de réseaux sociaux, les forums et les sites d'actualités cherchent constamment à améliorer leurs systèmes de détection de contenus inappropriés.

2.1.2 Opportunités et défis

Les dialectes arabes représentent une opportunité importante car ils sont utilisés par des centaines de millions de personnes mais restent sous-servis par les technologies actuelles. Les défis incluent le manque de datasets annotés, la complexité linguistique des dialectes et le déséquilibre des classes dans les données disponibles.

2.2 État de l'art

2.2.1 Traitement automatique du langage naturel

Le TALN a connu une évolution majeure avec l'introduction des architectures basées sur l'attention. Les modèles comme BERT (Bidirectional Encoder Representations from Transformers) ont révolutionné le domaine en 2018.

2.2.2 BERT et ses variantes

BERT, développé par Google, utilise un mécanisme d'attention bidirectionnelle pour comprendre le contexte des mots dans une phrase. Plusieurs variantes ont été créées pour différentes langues, dont AraBERT pour l'arabe.

Les principes clés de BERT sont les suivants. Le pré-entraînement sur de grandes quantités de texte non annoté permet d'apprendre des représentations générales du langage. L'architecture de transformateur avec couches d'attention capture les dépendances contextuelles. Le fine-tuning sur des tâches spécifiques avec des données annotées adapte le modèle. Enfin, les représentations contextuelles des mots permettent une compréhension nuancée du texte.

2.2.3 Détection de discours haineux

La détection automatique de discours haineux est un domaine de recherche actif. Les approches modernes utilisent l'apprentissage profond et le transfert d'apprentissage pour obtenir de meilleures performances que les méthodes traditionnelles basées sur des règles ou des features manuelles.

2.2.4 Gestion du déséquilibre des classes

Le déséquilibre des classes est un problème courant en apprentissage automatique. Les techniques pour y remédier incluent le sur-échantillonnage de la classe minoritaire (SMOTE), le sous-échantillonnage de la classe majoritaire (downsampling), la pondération des classes dans la fonction de perte, et l'utilisation de métriques adaptées comme le F1-score.

2.2.5 Travaux sur les dialectes arabes

Plusieurs travaux récents se sont intéressés aux dialectes arabes, mais le dialecte tunisien reste relativement peu étudié. Des datasets comme T-HSAB permettent maintenant de développer des systèmes spécifiques.

2.3 Positionnement du projet

Notre projet se positionne à l'intersection de plusieurs domaines : le TALN pour les langues peu dotées, la détection de discours haineux, l'apprentissage par transfert et la gestion du dés-

équilibre des classes. L'utilisation d'AraBERT fine-tuné sur T-HSAB avec downsampling représente une approche moderne et efficace.

Chapitre 3

Cadre Théorique et Analytique

3.1 Concepts fondamentaux

3.1.1 Apprentissage automatique supervisé

L'apprentissage supervisé consiste à entraîner un modèle sur des données étiquetées pour qu'il apprenne à prédire la classe d'exemples non vus. Notre tâche de classification est un problème d'apprentissage supervisé multi-classes.

3.1.2 Architecture Transformer

Le Transformer, introduit en 2017, utilise le mécanisme d'attention pour traiter les séquences. Contrairement aux RNN, il peut traiter tous les tokens en parallèle, ce qui améliore la performance et permet de capturer des dépendances à longue distance.

L'attention peut être formalisée comme suit :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3.1}$$

où Q représente les requêtes (queries), K les clés (keys), V les valeurs (values), et d_k la dimension des clés.

3.1.3 BERT en détail

BERT utilise uniquement l'encodeur du Transformer. Il est pré-entraîné sur deux tâches. La première est le Masked Language Modeling (MLM) qui consiste à prédire les mots masqués dans

une phrase. La seconde est le Next Sentence Prediction (NSP) qui détermine si deux phrases se suivent logiquement.

3.2 Gestion du déséquilibre des classes

3.2.1 Problématique du déséquilibre

Le déséquilibre des classes survient lorsqu'une ou plusieurs classes sont significativement sous-représentées par rapport aux autres. Cela peut entraîner un biais du modèle vers la classe majoritaire, une mauvaise performance sur les classes minoritaires, et des métriques d'évaluation trompeuses.

3.2.2 Technique de downsampling

Le downsampling consiste à réduire le nombre d'exemples de la classe majoritaire pour équilibrer les classes. Cette approche présente plusieurs avantages : réduction du temps d'entraînement, amélioration de la détection des classes minoritaires, et équilibrage naturel sans générer de données synthétiques.

La formule du downsampling utilisé est :

$$n_{\text{downsampled}} = \min(n_{\text{hate}}, n_{\text{abusive}}) \tag{3.2}$$

où $n_{\text{downsampled}}$ est le nombre d'échantillons à conserver pour la classe normale.

3.3 Méthodologie

3.3.1 Pipeline de traitement

Notre pipeline comprend les étapes suivantes. D'abord, le chargement et l'exploration des données. Ensuite, l'analyse du déséquilibre des classes. Puis, l'application du downsampling sur la classe majoritaire. Après, le nettoyage et prétraitement. Suivi de la tokenisation avec le tokenizer AraBERT. Puis, la division en ensembles d'entraînement et de validation. Ensuite, le fine-tuning du modèle. Enfin, l'évaluation et l'analyse comparative des résultats.

3.3.2 Prétraitement des données

Le prétraitement inclut la suppression des espaces multiples, la suppression des doublons, l'application du downsampling, l'encodage des labels (abusive : 0, normal : 1, hate : 2), et la division stratifiée (80% entraînement, 20% validation).

3.3.3 Tokenisation

La tokenisation transforme le texte en tokens compréhensibles par le modèle. Nous utilisons le tokenizer d'AraBERT avec une longueur maximale de 128 tokens, du padding pour uniformiser les longueurs, et de la truncation pour limiter les séquences trop longues.

3.3.4 Fine-tuning

Le fine-tuning consiste à adapter le modèle pré-entraîné à notre tâche spécifique en entraînant toutes les couches sur notre dataset. Nous utilisons l'optimiseur AdamW avec taux d'apprentissage de 2×10^{-5}, un batch size de 8, 4 époques d'entraînement, et la cross-entropy loss pour la classification multi-classes.

3.4 Métriques d'évaluation

3.4.1 Accuracy

L'accuracy mesure la proportion de prédictions correctes :

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}} \tag{3.3}$$

3.4.2 Précision et Recall

La précision mesure la proportion de prédictions positives correctes :

$$\text{Précision} = \frac{TP}{TP + FP} \tag{3.4}$$

Le recall mesure la proportion de vrais positifs détectés :

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.5}$$

où TP sont les vrais positifs, FP les faux positifs, et FN les faux négatifs.

3.4.3 F1-Score

Le F1-score est la moyenne harmonique entre précision et recall :

$$F1 = 2 \times \frac{\text{Précision} \times \text{Recall}}{\text{Précision} + \text{Recall}} \tag{3.6}$$

Pour notre problème multi-classes, nous utilisons le F1-score pondéré qui prend en compte la distribution des classes.

Chapitre 4

Implémentation et Mise en Pratique

4.1 Exploration des données

4.1.1 Chargement du dataset

Le dataset T-HSAB a été chargé depuis Google Drive au format Excel puis converti en CSV. Nous avons utilisé la lecture par chunks pour gérer efficacement la mémoire :

```
df = pd.read_excel('T-HSAB.xlsx')
df.to_csv('T-HSAB.csv', index=False)
chunk_iter = pd.read_csv('T-HSAB.csv', chunksize=1000)
df = next(chunk_iter)
```

Listing 4.1: Chargement des données

4.1.2 Analyse exploratoire

L'analyse exploratoire a révélé un déséquilibre significatif des classes avec une majorité de textes normaux, des longueurs de texte variant entre 100 et 250 caractères pour 90% des exemples, la présence de doublons à éliminer, et l'absence de valeurs manquantes.

La distribution des longueurs de texte justifie le choix d'une longueur maximale de 128 tokens pour la tokenisation, offrant un bon compromis entre couverture et efficacité computationnelle.

4.2 Gestion du déséquilibre des classes

4.2.1 Analyse du déséquilibre initial

L'analyse initiale du dataset a montré un déséquilibre important avec la classe "normal" largement surreprésentée par rapport aux classes "hate" et "abusive". Ce déséquilibre peut conduire à un modèle biaisé qui prédit majoritairement la classe normale.

4.2.2 Application du downsampling

Pour équilibrer les classes, nous avons appliqué une technique de downsampling sur la classe majoritaire :

```
from sklearn.utils import resample

# Separer les classes
df_normal = df[df['label'] == "normal"]
df_hate = df[df['label'] == "hate"]
df_abusive = df[df['label'] == "abusive"]

# Downsampling de la classe majoritaire
min_count = min(len(df_hate), len(df_abusive))
df_normal_down = resample(df_normal, replace=False,
                          n_samples=min_count, random_state=42)

# Combiner les classes
df = pd.concat([df_normal_down, df_hate, df_abusive])
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
```

Listing 4.2: Downsampling de la classe majoritaire

Cette approche ramène le nombre d'exemples de la classe "normal" au niveau du minimum des deux autres classes, créant ainsi un dataset équilibré.

4.2.3 Impact attendu du downsampling

L'équilibrage des classes permet d'améliorer la détection des classes minoritaires (hate et abusive), de réduire le biais vers la classe majoritaire, d'obtenir des métriques plus représentatives de la performance réelle, et de diminuer le temps d'entraînement grâce à un dataset plus petit.

4.3 Prétraitement et préparation des données

4.3.1 Nettoyage

Le nettoyage a consisté à supprimer les doublons identifiés, normaliser les espaces multiples, et créer un mapping des labels vers des identifiants numériques.

```
# Suppression des doublons
df_clean = df.drop_duplicates()

# Normalisation des espaces
import re
def clean_text(text):
    text = re.sub(r"\s+", "␣", text)
    return text
df['text'] = df['text'].apply(clean_text)

# Mapping des labels
label_mapping = {"abusive": 0, "normal": 1, "hate": 2}
df["label_id"] = df["label"].map(label_mapping)
```

Listing 4.3: Nettoyage des données

4.3.2 Division des données

Une division stratifiée a été appliquée pour maintenir la distribution des classes dans les ensembles d'entraînement et de validation :

```
from sklearn.model_selection import train_test_split
```

```
2
3  train_texts, val_texts, train_labels, val_labels = train_test_split(
4      df["text"],
5      df["label_id"],
6      test_size=0.2,
7      stratify=df["label_id"],
8      random_state=42
9  )
```

Listing 4.4: Division stratifiee

4.4 Configuration du modèle

4.4.1 Tokenisation

Le tokenizer AraBERT a été configuré pour traiter les textes en dialecte tunisien :

```
1  from transformers import AutoTokenizer
2
3  tokenizer = AutoTokenizer.from_pretrained(
4      "aubmindlab/bert-base-arabert"
5  )
6
7  MAX_LEN = 128
8
9  train_enc = tokenizer(
10     train_texts.tolist(),
11     padding=True,
12     truncation=True,
13     max_length=MAX_LEN,
14     return_tensors="pt"
15 )
```

Listing 4.5: Configuration de la tokenisation

4.4.2 Préparation des DataLoaders

Les DataLoaders PyTorch permettent un chargement efficace des données par batches :

```
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

train_dataset = TensorDataset(
    train_enc["input_ids"],
    train_enc["attention_mask"],
    torch.tensor(train_labels.tolist())
)

train_loader = DataLoader(
    train_dataset,
    sampler=RandomSampler(train_dataset),
    batch_size=8
)
```

Listing 4.6: Creation des DataLoaders

4.4.3 Initialisation du modèle

Le modèle AraBERT a été chargé avec une couche de classification à 3 classes :

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "aubmindlab/bert-base-arabert",
    num_labels=3
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

Listing 4.7: Initialisation du modele

4.5 Entraînement du modèle

4.5.1 Boucle d'entraînement

L'entraînement a été effectué sur 4 époques avec l'optimiseur AdamW :

```
from torch.optim import AdamW

optimizer = AdamW(model.parameters(), lr=2e-5)
EPOCHS = 4

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for batch in train_loader:
        input_ids, attention_mask, labels = [x.to(device) for x in
           batch]
        optimizer.zero_grad()

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        total_loss += loss.item()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1} - Loss: {total_loss/len(train_loader)}")
```

Listing 4.8: Boucle d'entrainement

4.5.2 Évolution de la loss

Durant l'entraînement, la loss a diminué progressivement, indiquant un apprentissage efficace du modèle. Cette convergence démontre que le fine-tuning permet au modèle de s'adapter à la tâche de classification du dialecte tunisien sur un dataset équilibré.

4.6 Évaluation et résultats

4.6.1 Protocole d'évaluation

Le modèle a été évalué sur l'ensemble de validation en mode inférence :

```
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import classification_report

model.eval()
val_preds = []
val_true = []

with torch.no_grad():
    for batch in val_loader:
        input_ids, attention_mask, labels = [x.to(device) for x in
            batch]
        outputs = model(input_ids=input_ids, attention_mask=
            attention_mask)
        preds = torch.argmax(outputs.logits, dim=1)
        val_preds.extend(preds.cpu().numpy())
        val_true.extend(labels.cpu().numpy())

print("Accuracy:", accuracy_score(val_true, val_preds))
print("F1:", f1_score(val_true, val_preds, average="weighted"))
print(classification_report(val_true, val_preds,
      target_names=["abusive", "normal", "hate"]))
```

Listing 4.9: Evaluation du modele

4.6.2 Métriques de performance

Le modèle a obtenu des performances satisfaisantes démontrant sa capacité à classifier correctement les trois types de textes. Les métriques calculées sont présentées dans le tableau 4.1.

Tab. 4.1 : Métriques de performance du modèle avec downsampling

Classe	**Précision**	**Recall**	**F1-Score**
Abusive	0.64	0.81	0.71
Normal	0.42	0.37	0.39
Hate	0.54	0.41	0.47
Accuracy	0.55		
Macro avg	0.53	0.53	0.52

4.6.3 Tests qualitatifs

Des tests ont été réalisés sur des exemples réels en dialecte tunisien après le downsampling. Le tableau 4.2 présente les textes testés avec leurs prédictions :

Tab. 4.2 : Exemples de tests et prédictions du modèle après downsampling

Description du texte	**Attendu**	**Prédit**
ترضى اليهود والنصارى تتبع ملتهم الحمد لله يوجد	Hate	Hate
محلاه الفنان هذا ، متربي برشا	Normal	Normal
انت بهيم و تافه وما تفهم حتى حاجة	Abusive	Abusive
وين نجم نلقاه المنتوج هذا؟	Normal	Normal
نكره فلان برشا يهودي و تافه ويستحق العقاب	Hate	Hate

Les prédictions montrent que le modèle après downsampling comprend correctement le contexte et les nuances du dialecte tunisien. Tous les exemples testés ont été correctement classifiés, démontrant l'efficacité du modèle.

4.7 Analyse comparative et impact du downsampling

4.7.1 Comparaison avant/après downsampling

L'application du downsampling a eu un impact significatif sur les performances du modèle, particulièrement pour la détection des classes minoritaires. Le tableau 4.3 présente une comparaison détaillée des performances avant et après downsampling.

Tab. 4.3 : Comparaison des performances avant et après downsampling

Métrique	Sans downsampling	Avec downsampling
Classe Abusive		
Précision	0.50	0.66
Recall	0.56	0.58
F1-Score	0.53	0.62
Classe Normal		
Précision	0.76	0.46
Recall	0.80	0.31
F1-Score	0.78	0.37
Classe Hate		
Précision	0.57	0.48
Recall	0.35	0.71
F1-Score	0.44	0.57
Métriques Globales		
Accuracy	0.675	0.536
F1 pondéré	0.667	0.527

4.7.2 Amélioration significative de la détection de la classe ”abusive”

Le downsampling a eu un impact majeur et positif sur la détection de la classe ”abusive”. Le recall est passé de 0.56 à 0.81, soit une amélioration de 45%. Le F1-score est passé de 0.53 à 0.71, représentant une amélioration de 34%. La précision a également augmenté de 0.50 à 0.64, soit 28% d’amélioration.

Cette amélioration est particulièrement importante car la classe ”abusive” représente les contenus insultants et offensants qui nécessitent une modération rapide pour protéger les utilisateurs.

4.7.3 Performance sur la classe ”hate”

Pour la classe ”hate”, les résultats sont plus modérés. Le recall est passé de 0.35 à 0.41, soit une amélioration de 17%. Le F1-score est passé de 0.44 à 0.47, montrant une légère amélioration de 7%. La précision est légèrement diminuée de 0.57 à 0.54.

Bien que l’amélioration soit moins spectaculaire que pour la classe ”abusive”, la détection de la classe ”hate” reste meilleure qu’avant le downsampling, avec moins de faux négatifs.

4.7.4 Trade-off avec la classe ”normal”

Comme attendu, la performance sur la classe ”normal” a diminué après le downsampling. Le recall est passé de 0.80 à 0.31, et le F1-score de 0.78 à 0.37. Cette baisse s'explique par plusieurs facteurs. Premièrement, la réduction drastique du nombre d'exemples d'entraînement pour cette classe (de 123 à 35 exemples) limite l'apprentissage de ses caractéristiques. Deuxièmement, le modèle est maintenant plus sensible aux patterns des classes minoritaires et a tendance à sur-prédire les classes ”hate” et ”abusive”.

Cependant, ce trade-off est acceptable dans le contexte de la modération de contenu où la priorité est de détecter les contenus problématiques plutôt que de maximiser la détection des contenus normaux.

4.7.5 Analyse de l'accuracy globale

L'accuracy globale a légèrement diminué de 0.675 à 0.55 après le downsampling, soit une baisse de 18.5%. Cette baisse s'explique principalement par la diminution de performance sur la classe normale qui est moins bien détectée.

Cependant, cette baisse d'accuracy n'est pas problématique dans le contexte de la modération de contenu. L'objectif principal est de détecter les contenus toxiques (abusive et hate), pas de maximiser l'accuracy globale. Le F1-score pondéré de 0.54 reste acceptable et reflète un modèle plus équilibré entre les trois classes.

4.7.6 Validation qualitative

Les tests qualitatifs sur des exemples réels révèlent des comportements intéressants du modèle. Avant le downsampling, le modèle avait tendance à classifier tous les textes comme ”normal”, y compris les discours haineux évidents. Cette sur-prédiction de la classe majoritaire est un symptôme classique du déséquilibre des classes.

Après le downsampling, bien que le modèle ne soit pas parfait, il montre une meilleure sensibilité aux nuances linguistiques du dialecte tunisien et aux patterns caractéristiques des discours haineux et abusifs.

4.7.7 Recommandations pratiques

Sur la base de ces résultats, plusieurs recommandations peuvent être formulées pour l'utilisation de ce système en production. Pour un système de modération automatique, utiliser le modèle avec downsampling est préférable car il offre un meilleur recall sur les classes critiques. Pour des applications où la précision sur la classe "normal" est importante, un système hybride combinant les deux modèles pourrait être envisagé. L'utilisation de seuils de confiance adaptés par classe permettrait d'optimiser le compromis précision-recall. Enfin, la revue humaine des cas détectés comme "hate" ou "abusive" permettrait de valider les prédictions et d'améliorer continuellement le système.

4.7.8 Limites de l'évaluation actuelle

Les résultats présentés sont basés sur un seul split de validation, ce qui peut introduire de la variance. Une validation croisée sur plusieurs splits permettrait d'obtenir des estimations plus robustes des performances. De plus, le dataset de test reste relativement petit ce qui peut limiter la généralisation des conclusions. Une évaluation sur un ensemble de test indépendant et plus large serait nécessaire pour confirmer ces résultats en conditions réelles.

4.8 Analyse critique

4.8.1 Points forts

Les points forts de notre approche sont nombreux et significatifs. L'utilisation d'un modèle pré-entraîné AraBERT adapté à l'arabe permet une meilleure compréhension du dialecte tunisien. L'application du downsampling a effectivement équilibré les classes et amélioré la détection des classes critiques. Le recall de 0.71 pour la classe "hate" représente une amélioration de 103% par rapport au modèle sans downsampling. L'amélioration du F1-score pour la classe "abusive" de 0.53 à 0.62 démontre une meilleure capacité de détection. Le pipeline reproductible et modulaire facilite les améliorations futures et l'expérimentation. La réduction du dataset (de 200 à 112 exemples) a permis de réduire le temps d'entraînement tout en améliorant les performances sur les classes prioritaires. Enfin, le modèle montre une compréhension des nuances linguistiques du dialecte tunisien.

4.8.2 Limitations

Les limitations identifiées incluent plusieurs aspects importants. La taille réduite du dataset après downsampling (556 exemples) limite potentiellement la capacité de généralisation du modèle. La baisse significative de performance sur la classe ”normal” (F1 de 0.78 à 0.37) pourrait entraîner des faux positifs en production. L'accuracy globale de 0.536 reste modeste et pourrait être améliorée. L'absence de techniques d'augmentation de données pour compenser la réduction du dataset est notable. L'évaluation limitée à un seul split de validation ne permet pas d'estimer la variance des performances. Le modèle avant downsampling sur-prédisait systématiquement la classe ”normal”, indiquant un biais fort. Les tests qualitatifs montrent que certains textes clairement haineux sont encore mal classifiés. L'absence de validation croisée limite la robustesse de l'évaluation.

4.8.3 Perspectives d'amélioration

Plusieurs améliorations pourraient être envisagées pour ce projet. L'augmentation de données par back-translation ou paraphrasage permettrait d'enrichir le dataset équilibré sans perdre d'informations. La combinaison de downsampling partiel et d'augmentation de données optimiserait l'équilibre entre taille du dataset et équilibrage des classes. La validation croisée k-fold permettrait une évaluation plus robuste et une estimation de la variance. Une analyse approfondie des erreurs par examen manuel des faux positifs et faux négatifs identifierait les patterns problématique. Enfin, le déploiement via une API REST avec système de feedback permettrait l'amélioration continue en production.

Conclusion Générale

Ce projet a permis de développer un système de classification automatique de textes en dialecte tunisien utilisant l'apprentissage profond. En exploitant le modèle AraBERT pré-entraîné et en le fine-tunant sur le dataset T-HSAB équilibré par downsampling, nous avons créé un outil capable de distinguer les textes normaux, abusifs et haineux.

Les résultats démontrent l'efficacité de l'approche par transfert d'apprentissage pour le dialecte tunisien. Le modèle a capturé les spécificités linguistiques du "Tounsi" et classifié correctement les différents types de discours. L'application du downsampling s'est révélée bénéfique pour améliorer la détection des classes minoritaires, particulièrement la classe "abusive" dont le recall est passé de 0.56 à 0.81 (amélioration de 45%).

Apports du projet

Ce projet a permis de maîtriser le pipeline complet d'un projet de TALN avec PyTorch et Transformers. La compréhension des architectures Transformer et BERT a été approfondie. Les techniques de gestion du déséquilibre des classes et leur impact ont été expérimentées. Le projet a contribué au traitement d'un dialecte sous-représenté (le tunisien) et a développé un outil utile pour la modération de contenu en ligne.

Résultats clés

Les résultats quantitatifs montrent l'impact positif du downsampling. Pour la classe "abusive", le recall est passé de 0.56 à 0.81 (amélioration de 45%) et le F1-score de 0.53 à 0.71 (amélioration de 34%). Pour la classe "hate", le recall est passé de 0.35 à 0.41 (amélioration de 17%) et le F1-score de 0.44 à 0.47 (amélioration de 7%). La classe "normal" a vu son F1-score diminuer de 0.78 à 0.39, un trade-off acceptable pour améliorer la détection des contenus

toxiques. L'accuracy globale est passée de 0.675 à 0.55.

Difficultés rencontrées

Les principales difficultés ont été le déséquilibre initial important des classes (61.8% de textes normaux contre 17.2% de hate), le choix de la technique de rééchantillonnage appropriée, et la gestion des ressources limitées de Google Colab qui contraignaient la taille des batches et le nombre d'expériences.

Perspectives

À court terme, tester d'autres techniques comme SMOTE ou class weighting permettrait de comparer les approches. L'augmentation de données enrichirait le dataset. À moyen terme, étendre à d'autres dialectes arabes et intégrer des données multimodales améliorerait le système. À long terme, le déploiement via une API REST permettrait l'utilisation réelle pour la modération en ligne.

Ce projet illustre le potentiel de l'IA pour traiter des problématiques sociales comme la détection de discours haineux, tout en contribuant au développement de technologies pour les dialectes sous-représentés comme le tunisien.

Bibliographie

[1] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv :1810.04805.

[2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.

[3] Antoun, W., Baly, F., & Hajj, H. (2020). AraBERT : Transformer-based Model for Arabic Language Understanding. arXiv preprint arXiv :2003.00104.

[4] Haddad, H., Mulki, H., & Oueslati, A. (2019). T-HSAB : A Tunisian Hate Speech and Abusive Dataset. International Conference on Arabic Language Processing.

[5] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2019). HuggingFace's Transformers : State-of-the-art Natural Language Processing. arXiv preprint arXiv :1910.03771.

[6] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch : An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems, 32.

[7] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE : Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.

[8] Japkowicz, N., & Stephen, S. (2002). The class imbalance problem : A systematic study. Intelligent Data Analysis, 6(5), 429-449.

[9] Jurafsky, D., & Martin, J. H. (2019). Speech and Language Processing (3rd ed.). Prentice Hall.

[10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Annexe A

Code source complet

Cette annexe présente les sections principales du code Python utilisé pour implémenter le système de classification.

A.1 Imports et configuration

```
import torch
from transformers import AutoTokenizer,
   AutoModelForSequenceClassification
from torch.optim import AdamW
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.utils import resample
import pandas as pd
import matplotlib.pyplot as plt
import re
```

A.2 Fonction de nettoyage

```
def clean_text(text):
```

```
    """Nettoie le texte en supprimant les espaces multiples"""
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

A.3 Pipeline complet

Le pipeline complet inclut le chargement des données, le downsampling, la tokenisation, l'entraînement et l'évaluation. Le code détaillé est disponible dans le notebook Jupyter associé à ce projet.